A Real-Time Traffic Digital Twin from Public CCTV: Vision-Based Detection, Map-Anchored Localization, and Self-Calibrating Speed Estimation

Minsoo Ahn
Department of Computer Science
Stony Brook University
minsoo.ahn@stonybrook.edu

Dahyun Kwon
Department of Computer Science
Stony Brook University
dahyun.kwon@stonybrook.edu

***Abstract*—We present a real-time traffic digital twin that converts live public CCTV feeds into an interactive, map-anchored view of road traffic. From a single clicked camera the system streams HLS video, detects vehicles with YOLO26 [1]—an NMS-free end-to-end detector (YOLOv8 [2] retained as legacy fallback)—and tracks them with a pluggable multi-object tracker (BoxMOT), and converts each vehicle's pixel position to a geographic coordinate through a perspective homography. Because public CCTV cameras are uncalibrated and their intrinsics are unknown, accurate ground-plane mapping and speed estimation are the central challenges. We address them with three complementary mechanisms: (i) a two-stage *curved* transform that maps pixels along the true road centreline obtained from the national node-link road network rather than a flat plane; (ii) a five-stage speed estimator combining a physics-based outlier guard, least-squares regression over a sliding window, and per-track exponential smoothing; and (iii) an online self-calibration loop that compares the measured speed against the official ITS segment speed and learns a per-camera correction factor. The system additionally provides multi-camera background monitoring, camera-level congestion clustering, and a SQLite time-series history with charting. We describe the architecture and the key algorithms, and report latency, throughput, tracking-stability, and speed metrics produced by a measurement harness over live streams.**

***Index Terms*—digital twin, intelligent transportation systems, object detection, multi-object tracking, homography, perspective transform, speed estimation, real-time visualization.**

I. Introduction

Cities expose thousands of public traffic cameras, but the raw feeds are difficult to interpret at scale: an operator watching a video grid cannot read off vehicle counts, speeds, or where congestion is forming on a map. A *digital twin*—a live, geo-referenced model of the road state—turns those feeds into actionable information. The technical obstacle is that public CCTV cameras are *uncalibrated*: their height, pitch, focal length, and exact heading are unknown, so projecting an image detection onto the ground plane (and therefore measuring distance and speed) is ill-posed without per-camera calibration.

This paper describes a system that ingests Korean ITS CCTV streams and produces a real-time map-anchored twin. Our contributions, from a systems and algorithms standpoint, are:

- **Map-anchored localization.** We snap each camera onto the national node-link road network, extract the road centreline, and map pixels along the *curve* of the road in two stages, rather than assuming a flat homographic plane. This keeps vehicle markers on the road even on bends.
- **Robust speed estimation without ground truth.** A five-stage filter (duplicate skip, physics jump guard, sliding-window least-squares regression, per-track exponential smoothing with spike rejection, and scale correction) produces stable speeds despite detection jitter and tracker ID churn.
- **Online self-calibration.** Because absolute scale is unknowable from an uncalibrated camera, we learn a per-camera `speed_scale` by comparing our measured average against the ITS official segment speed, with convergence and volatility guards.
- **A complete, resilient pipeline.** Automatic lane-based calibration, manual 4-point calibration, dual-path inference (browser canvas and server-side HLS) over a shared detector, multi-camera background monitoring, congestion clustering, and a historical analytics store.

The rest of the paper is organized as follows. Section II reviews related work. Section III presents the system architecture. Section IV details the algorithms. Section V reports the measurements. Section VI discusses limitations, and Section VII concludes.

II. Related Work

Object detection and tracking. Modern traffic vision builds on real-time detectors; we use YOLO26 [1] as the default detector—an NMS-free end-to-end architecture that produces more consistent per-frame bounding-box positions than NMS-based models such as YOLOv8 [2], reducing the ground-contact jitter that propagates into speed estimates. It also simplifies the TensorRT FP16 engine graph (no NMS layer) [3], lowering inference latency and its variance; YOLOv8 is retained as a legacy fallback. For tracking we use the BoxMOT framework [4], which exposes ByteTrack [5], BoT-SORT [6], and OC-SORT [7]; appearance re-identification uses OSNet [8]. Trackers without appearance ReID suffer ID switches under occlusion, which we mitigate with a lightweight position-based ID stabilizer.

Camera-to-ground mapping. Mapping image points to a ground plane is a homography problem [9], typically solved from four correspondences with OpenCV [10]. For *uncalibrated* cameras, vanishing-point and lane-geometry cues are commonly used to recover pose. Our work combines a 4-point manual mode, a vanishing-point/lane auto-calibration, and—novel in this context—a road-centreline-following transform driven by an external road-network dataset [11].

ITS dashboards and data. National ITS portals [12] publish CCTV streams and aggregated segment speeds, but typically as separate views. We *fuse* the segment speed back into the pipeline as a calibration signal. Congestion grouping uses density-based clustering [13] and convex hulls [14]; service levels follow the Highway Capacity Manual [15]. The visualization uses deck.gl [16] over MapLibre [17], served by FastAPI [18].

III. System Design and Architecture

The backend (FastAPI/Uvicorn) and a React/deck.gl frontend communicate over REST and WebSocket. Two inference paths can produce per-frame analytics and *share a single detector*; a connection counter coordinates them so that the multi-object tracker is never driven by two interleaved frame streams (Fig. 1).

Browser path (`/ws/detect`). The frontend plays the HLS stream through a CORS proxy, captures canvas frames (JPEG, $\leq$640 px, $\approx$33 ms interval, at most two in flight), and sends them to the backend, which runs detection, tracking, localization, and analytics, then returns an annotated frame and broadcasts the analytics JSON to all clients.

Server path (`live_loop`). The backend can also read the HLS stream directly with OpenCV/FFmpeg and run the same pipeline. It is gated twice: it skips tracking while a browser detection client is active (to protect the shared tracker), and it skips all GPU work when no one is watching (the frontend reports tab visibility).

Background services. Periodic loops refresh expiring HLS tokens (30 min), poll the ITS segment speed for self-calibration (5 min), and snapshot all monitored cameras into a SQLite history while recomputing congestion clusters (30 s). The resulting map view is shown in Fig. 2.

IV. Implementation

A. Detection and Tracking

The detector loads the fastest available backend—TensorRT FP16 engine, then ONNX Runtime, then PyTorch—and exports `.pt`$\to$`.engine` on first run when a GPU is present. A tracker *tier* is chosen by GPU memory: ByteTrack (CPU/low-VRAM), OC-SORT (occlusion-robust, no ReID), or BoT-SORT/DeepOC-SORT with OSNet appearance ReID on larger GPUs. For the non-ReID tiers we add two robustness layers: `_dedup_tracks` merges duplicate boxes (IoU $>$ 0.3 or centre distance $<$ 40 px, keeping the older ID), and an `IDStabilizer` re-binds a vehicle's previous ID after a brief miss using nearest-centre matching ($\leq$80 px) with a two-pass scheme and stale-mapping purge so display IDs stay stable and compact. Fig. 3 shows the detection and tracking overlay.

Algorithm 1 Two-stage curved pixel$\to$GPS transform

Require: pixel (u, v); metric homography H_m; centreline *road_pts*; *snap_along*; direction sign s
1: $(x_m, y_m) \leftarrow H_m \cdot (u, v)$
2: $(dx, dy) \leftarrow (x_m, y_m) - \mathit{snap}_m$ ▷ ENU from snap
3: $d_\parallel \leftarrow dx \sin b + dy \cos b$; $d_\perp \leftarrow dx \cos b - dy \sin b$
4: $\mathit{arc} \leftarrow \mathit{snap\_along} + s \cdot \max(0, d_\parallel)$
5: $(\mathit{lat}, \mathit{lon}) \leftarrow \textsc{InterpCenterline}(\mathit{arc})$
6: $b_\ell \leftarrow \textsc{RoadBearingAt}(\mathit{arc})$
7: **apply** $d_\perp$ perpendicular to b_ℓ **return** $(\mathit{lat}, \mathit{lon})$

B. Map-Anchored Localization

On camera switch we query the national node-link network: we snap the camera position perpendicularly onto the nearest road polyline, extract $\pm$150 m of centreline, and—because each direction is stored as a separate one-way link—we locate the reverse carriageway and average the two snaps (when 2–40 m apart) to recover the true road centre. The chosen road's speed limit, lane count, and width (lanes $\times$ 2 $\times$ lane-width) are attached to the camera; OSM Overpass provides a width refinement when available.

Given the centreline, the *two-stage curved transform* maps a pixel (u, v) to GPS along the road (Algorithm 1): stage one maps the pixel to local metres via the metric homography; stage two decomposes the displacement from the snap point into along-road and lateral components, advances that arc length along the centreline, interpolates the on-road position, and re-applies the lateral offset perpendicular to the *local* road bearing. This follows real road curvature, unlike a single flat homography.

When no manual calibration exists, an automatic procedure estimates a homography from one frame: Canny edges in the lower image, Hough line filtering, multi-row sampling of left/right road edges, least-squares edge fits, vanishing-point intersection, a pitch/height estimate from a pinhole model, and a trapezoid-to-GPS homography. Direction (which way the camera looks along the road) is resolved by matching the *image* road-curve sign against the *map* curve sign, falling back to a vanishing-point heading test. A manual 4-point mode remains available for maximum accuracy. Fig. 4 shows an automatic calibration result.

The primary auto-calibration path is a road-model *pose solver* (`camera_pose.solve_pose`): the sampled lane edges, vanishing point, and NodeLink road width are fitted to a four-parameter pinhole-camera model (H, pitch, yaw, lateral offset x_0) via `scipy.optimize.least_squares` (soft-L1 loss). A per-row reprojection residual gates the output ($<$8 px); on success the solved pose is converted to four image–GPS corner pairs and used to build both homographies. The pose is then persisted per camera to `camera_pose.json` so

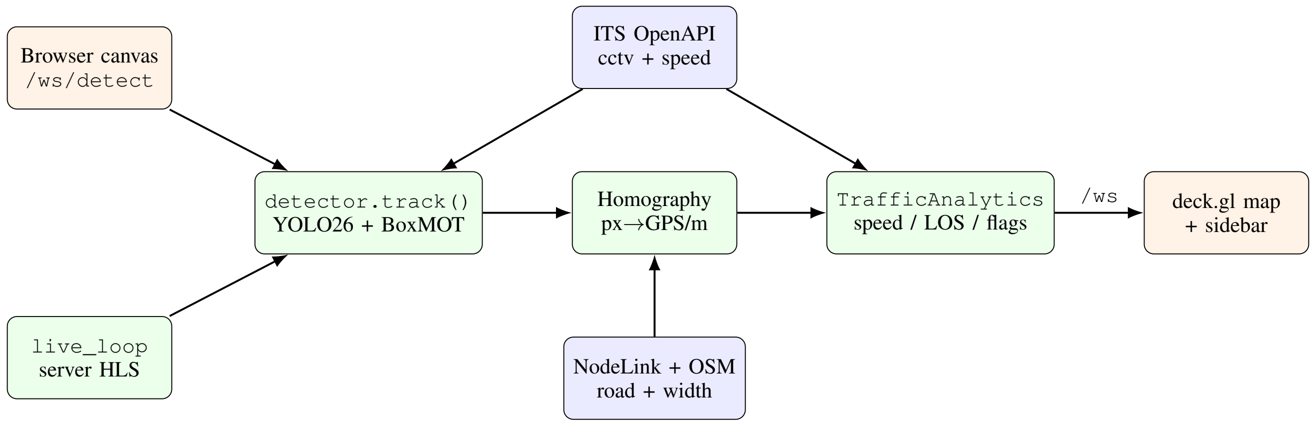

Fig. 1. System architecture. The browser path (`/ws/detect`) and the server path (`live_loop`) share one detector; a counter prevents concurrent tracker calls. External data (ITS CCTV/speed, NodeLink, OSM) feeds localization and self-calibration.

[figure placeholder: `cctv_map.png`]
Map view with status-coloured CCTV icons and FOV polygon

Fig. 2. Map view: public CCTV cameras as status-coloured icons on the vector map; the selected camera shows its field-of-view polygon and the located vehicles.

[figure placeholder: `yolo_detect.png`]
YOLO detection and multi-object tracking overlay

Fig. 3. Detection and tracking on a live CCTV frame: bounding boxes, vehicle class, and stabilized track IDs.

[figure placeholder: `auto_calib.png`]
Automatic lane-based calibration result

Fig. 4. Automatic calibration: detected lane edges and the estimated vanishing point yield the ground homography and the field-of-view footprint.

each session refines the prior, with a cold-start fallback chain (saved prior → vehicle-bbox rough pose → GPS-grid). The trapezoid heuristic above serves only as the fallback when the residual exceeds the quality gate or lane detection fails.

C. Speed Estimation

Speed is distance over time, and both terms are error-prone: homography distance is noisy at the far field, and HLS buffering makes naive frame timing wrong. We use a monotonic clock derived from stream presentation timestamps (falling back to wall-clock), and the five-stage estimator of Algorithm 2 with the constants in Table I.

Algorithm 2 Per-track speed estimation (`analytics._speed`)

Require: window W, prior EMA v_e, $\sigma \leftarrow$ `speed_scale`

if duplicate position **then** skip ▷ dedup
if *step* $> (V_{\max}/3.6\sigma)\,dt \cdot 1.5$ **then** drop sample, keep W ▷ jump guard
if $dt > 2\,\mathrm{s}$ **then** clear W ▷ track gap
fit $\hat{v}$ by OLS on W ($0.7\,\mathrm{s}$); **if** $\mathrm{disp}(W) < 0.5\,\mathrm{m}$ **then** $\hat{v} \leftarrow 0$ ▷ OLS
$v_e \leftarrow (1-\alpha)v_e + \alpha\hat{v}\sigma$, α=0.35; reject spike; decay on stop ▷ EMA
output v_e; *is_speeding* $\leftarrow v_e > 1.10 \cdot$ *limit* ▷ output

D. Online Self-Calibration

The absolute distance scale of an uncalibrated camera is unknowable, so a residual scale error persists after geometric calibration. Every five minutes we fetch the ITS official segment speed near the camera and update a per-camera factor `speed_scale`:

$$\textit{target} = \textit{scale} \cdot \frac{v_{\mathrm{ITS}}}{\bar{v}_{\mathrm{ours}}}, \quad \textit{scale} \leftarrow (1-\alpha)\,\textit{scale} + \alpha\,\textit{target}, \tag{1}$$

clamped to $[0.3, 5.0]$, using a 10-minute rolling average ($\geq$50 samples), a coefficient-of-variation guard ($\leq$0.4) to skip volatile traffic, and a learning rate that drops from 0.5 to 0.05 once converged (three consecutive $<$1 % updates). The converged factor is persisted per camera.

TABLE I
KEY SPEED/ANALYTICS CONSTANTS (`CONFIG.PY`).

Parameter	Value
Speed window	0.7 s (W=round(0.7s×FPS))
EMA α	0.35
Jitter threshold	0.5 m
Min speed (else 0)	5 km/h
Max plausible $V_{\max}$	180 km/h
GC grace	30 frames
Bottleneck dwell	150 frames (≈5 s)
Parked dwell	300 frames (≈10 s)
Speeding threshold	$v_e > 1.10 \times$ limit
LOS thresholds A–E	≤3,6,9,12,15 (else F)

[figure placeholder: `congestion_map.png`]
Congestion overlay: CCTV icons recoloured by status

Fig. 5. Camera-level congestion: monitored cameras are recoloured (normal/busy/congested) and adjacent congested cameras are grouped into a severity-shaded region.

E. Aggregation, Congestion, and History

A horizontal line zone yields In/Out counts and per-vehicle direction. Dwell counting flags bottlenecks and parked vehicles (the latter remembered by pixel position so they survive ID changes), and a Level-of-Service grade is assigned from the active vehicle count. Background monitoring polls additional cameras every 8 s with detection only; busy/congested cameras are clustered with a haversine density-based scan (ε=500 m) and outlined with a convex hull. A WAL-mode SQLite store records 30 s snapshots (14-day retention) and serves bucketed time series, peak-time detection, and CSV export. Fig. 5 shows the congestion overlay.

F. Visualization

The deck.gl map layers congestion polygons, vehicle trails, the FOV polygon, status-coloured camera icons, and vehicle markers. The FOV polygon uses one of three strategies in priority order: the manually clicked GPS ring; a curved corridor that follows the road centreline (mirroring the backend transform); or, when uncalibrated, a default trapezoid.

The sidebar's vehicle list provides *direction-filter tabs* (All / Inbound / Outbound) that filter the displayed rows by the LineZone-derived direction; the per-direction vehicle count is shown in each tab badge, and the min/avg/max speed summary is recomputed from the filtered set. Two collapsible cards (Auto Calibration Estimate, ITS Speed Comparison) include an info button that opens a modal explanation of each metric, adapting to the selected map theme.

TABLE II
PER-STAGE LATENCY (MS) AND THROUGHPUT. FILL FROM `EVAL_LATENCY.CSV`.

Stage	Mean	Median	p95
YOLO (isolated)	—	—	—
Track (YOLO+BoxMOT)	—	—	—
Transform	—	—	—
Analytics	—	—	—
End-to-end	—	—	—
Throughput: — FPS			

TABLE III
TRACKING STABILITY AND SPEED DISTRIBUTION. FILL FROM `EVAL_TRACKING.CSV` / `EVAL_SPEED.CSV`.

Metric	Value
Frames processed	—
Unique tracks	—
ID appearances (switch proxy)	—
Mean track lifetime (frames)	—
Median measured speed (km/h)	—
% moving observations	—

V. EVALUATION

Measurements are produced from the real pipeline in two ways. While the system runs (`make dev`), an in-process collector records per-frame stage latency, tracking, speed, and detection statistics and flushes `eval_*.csv` and `eval_summary.json` every 30 s (also on demand via `GET /eval/report`); an offline harness (`backend/evaluate.py`) reproduces the same metrics over a fixed clip for controlled benchmarks. Numbers below are produced on the target hardware.

Setup: model `[YOLO26m]`, backend `[TensorRT/ONNX/PyTorch]`, tracker `[tier]`, GPU `[device]`, source `[camera/clip]`, `[N]` frames.

Speed accuracy. Because no per-vehicle ground truth exists for public CCTV, we assess speed against the ITS official segment speed for the same camera and time window. Table IV compares the measured 10-minute average against ITS and reports the learned scale factor and convergence.

VI. DISCUSSION AND LIMITATIONS

The homography is accurate inside the calibration quadrilateral and extrapolates with growing error toward the far field; we mitigate this with the dual-matrix calibration, the regression window, lane-based auto-calibration, and the ITS `speed_scale`, but a residual error remains on poorly observed cameras. Automatic calibration assumes a focal length of 1.2× image height (unknown intrinsics), giving a ±15–20 % scale uncertainty per FoV mismatch, and fails when lane markings are not visible (night, rain, dense traffic), falling back to an approximate grid. Road width is inferred from lane count and assumed bidirectional, so one-way roads are over-wide. The nearest node-link road is occasionally

TABLE IV
MEASURED VS. ITS SEGMENT SPEED (PER CAMERA). FILL FROM THE LIVE OUR_AVG_KPH/ITS_SPEED_KPH/SPEED_ERROR_PCT BROADCAST AND SPEED_SCALE.JSON.

Camera	Ours (km/h)	ITS (km/h)	Err %	scale	conv.
—	—	—	—	—	—
—	—	—	—	—	—

the wrong one (e.g., a camera near both a national route and an expressway), which displaces the FOV polygon and vehicle GPS; the CCTV-name road-hint reduces but does not eliminate this. Finally, congestion is clustered at the camera level because background cameras expose only counts, not per-vehicle speeds. A structural limitation is the system's dependence on national ITS infrastructure: camera streams and segment speeds are fetched from a government-operated open API [12] whose availability and authentication policy are outside the operator's control and may be restricted or discontinued. Camera placement and viewing angle are fixed by road authorities and cannot be adjusted; some installations are positioned too far or too close to capture a useful road segment, and extreme viewpoints cannot be recovered through calibration. Adverse conditions—low illumination, rain, lens contamination, and hardware faults—can suppress the visual signal entirely, leaving the pipeline with no basis for detection or calibration. These constraints are inherent to re-using existing infrastructure without modification.

VII. CONCLUSION

We built a real-time traffic digital twin that localizes vehicles on a map from uncalibrated public CCTV and estimates their speed without ground truth. The key ideas are a road-centreline-following two-stage transform, a robust five-stage speed estimator, and an online self-calibration loop driven by official ITS segment speeds, wrapped in a resilient dual-path pipeline with congestion analytics and a historical store. The approach makes raw municipal camera feeds directly usable for situational awareness, and its self-calibration removes the main barrier—per-camera manual setup—to deploying across many cameras.

A distinguishing property is that the system requires *no GPS instrumentation on vehicles*: conventional traffic monitoring relies on embedded GPS loggers or dedicated roadside sensors, both of which demand significant infrastructure investment. By re-purposing cameras that road authorities already operate, the system tracks multiple vehicle classes simultaneously across multiple views with no additional hardware cost. As public CCTV coverage continues to expand and object detectors advance, this class of vision-only traffic twin becomes increasingly practical at scale. With wider network integration, the same localization principle—positioning objects through perspective projection rather than satellite signal—could extend to GPS-denied environments such as tunnels, underground parking, and indoor logistics facilities; the per-camera track-ID scheme further enables cross-camera vehicle re-identification for a range of fleet management and security applications.

REFERENCES

[1] Ultralytics, "YOLO26: Key architectural enhancements and performance benchmarking for real-time object detection," *arXiv preprint arXiv:2509.25164*, 2025. [Online]. Available: https://arxiv.org/abs/2509.25164

[2] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," https://github.com/ultralytics/ultralytics, 2023, open-source object detection and tracking framework.

[3] NVIDIA, "TensorRT: High-performance deep learning inference optimizer and runtime," https://developer.nvidia.com/tensorrt, 2023.

[4] M. Broström, "BoxMOT: Pluggable SOTA multi-object tracking modules," https://github.com/mikel-brostrom/boxmot, 2023.

[5] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang, "ByteTrack: Multi-object tracking by associating every detection box," in *Proc. European Conf. on Computer Vision (ECCV)*, 2022, pp. 1–21.

[6] N. Aharon, R. Orfaig, and B.-Z. Bobrovsky, "BoT-SORT: Robust associations multi-pedestrian tracking," *arXiv preprint arXiv:2206.14651*, 2022.

[7] J. Cao, J. Pang, X. Weng, R. Khirodkar, and K. Kitani, "Observation-centric SORT: Rethinking SORT for robust multi-object tracking," in *Proc. IEEE/CVF Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2023.

[8] K. Zhou, Y. Yang, A. Cavallaro, and T. Xiang, "Omni-scale feature learning for person re-identification," *Proc. IEEE/CVF Int. Conf. on Computer Vision (ICCV)*, 2019.

[9] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.

[10] G. Bradski, "The OpenCV library," *Dr. Dobb's Journal of Software Tools*, 2000.

[11] Korea Transport Database (KTDB) / Ministry of Land, Infrastructure and Transport, "Standard node-link (MOCT) road network dataset," https://www.its.go.kr/nodelink, 2023.

[12] National Transport Information Center (ITS), Republic of Korea, "ITS open API: Real-time CCTV and traffic information," https://openapi.its.go.kr, 2023.

[13] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," in *Proc. 2nd Int. Conf. on Knowledge Discovery and Data Mining (KDD)*, 1996, pp. 226–231.

[14] A. M. Andrew, "Another efficient algorithm for convex hulls in two dimensions," *Information Processing Letters*, vol. 9, no. 5, pp. 216–219, 1979.

[15] Transportation Research Board, "Highway capacity manual (HCM)," *National Research Council, Washington, D.C.*, 2016.

[16] vis.gl / Open Visualization, "deck.gl: WebGL2-powered geospatial visualization framework," https://deck.gl, 2023.

[17] MapLibre Contributors, "MapLibre GL JS," https://maplibre.org, 2023.

[18] S. Ramírez, "FastAPI," https://fastapi.tiangolo.com, 2018.